Chapter 6: Writing Your First CodeQL Queries

Download this chapter as a PDF for offline reading:
Chapter-06-Writing-Your-First-CodeQL-Queries.pdf

You now know how to write predicates and organize them into libraries. This chapter shows how to turn that logic into complete CodeQL queries: the metadata that describes a query, the structure of simple alert queries, and a first look at path queries.

Metadata

Every `.ql` file that you intend to run as a query should begin with a QLDoc comment containing metadata. The metadata tells the CodeQL CLI, the VS Code extension, and GitHub code scanning how to interpret the results and how to display them.

The metadata is written as a block comment that starts with `/**` and ends with `*/`. Each property appears on its own line beginning with `@`.

Required properties

Property	Purpose
@name	Short human-readable title shown in results
@description	Longer explanation of what the query finds
@kind	Query type: problem, path-problem, metric, or diagnostic
@id	Unique identifier in the form scope/name (for example mustlearn/js-empty-functions)

Optional but useful properties

Property	Values	Purpose
@tags	space-separated list	Categories such as security, maintainability, correctness
@problem.severity	error, warning, recommendation	How serious the issue is
@precision	very-high, high, medium, low	Expected true-positive rate
@security-severity	number 0.0–10.0	Severity score for security queries

`@kind` is required when you run the query with `codeql database analyze`. It is also required if you want the VS Code extension to display results as alerts or paths.

Example metadata for a simple alert query:

```
/**
 * @name Empty functions
 * @description Finds functions that contain no statements.
 * @kind problem
 * @id mustlearn/js-empty-functions
 * @tags maintainability
 * @problem.severity recommendation
 * @precision high
 */
```

The full set of properties and style guidance is documented in the official CodeQL query metadata style guide.

Alert Queries

An alert query (`@kind problem`) reports a single code location together with a message. This is the most common query type for custom checks.

The `select` clause must have exactly two columns:

```
select element, message
```

- `element` is the AST node or other location where the alert is displayed.
- `message` is a string (or a string expression) that describes the problem.

Complete example

The following query finds empty functions in a JavaScript database. It lives in the companion repository at `examples/chapter06/empty-functions.ql`:

```
/**
 * @name Empty functions
 * @description Finds functions that contain no statements.
 * @kind problem
 * @id mustlearn/js-empty-functions
 * @tags maintainability
 * @problem.severity recommendation
 * @precision high
 */
import javascript

from Function f
where not exists(f.getAStmt())
select f, "This function is empty."
```

Run it against a database with:

```
codeql query run --database=my-js-db empty-functions.ql
```

For production or CI use, prefer `codeql database analyze`, which produces SARIF and respects the metadata.

Metric queries

A related type is `@kind metric`. Metric queries return numeric measurements rather than alerts. The `select` clause can return a single number or an entity plus a number.

```
/**
 * @name Number of functions
 * @description Counts the number of functions in the database.
 * @kind metric
 * @id mustlearn/js-function-count
 * @tags maintainability
 */
import javascript

select count(Function f)
```

(See `examples/chapter06/function-count.ql`.)

Path Queries Preview

A path query (`@kind path-problem`) reports an alert together with a sequence of code locations that explain how the problem arises. The classic use case is data-flow or taint tracking from a source to a sink, but the same mechanism can also show control-flow or other paths.

A path query has three extra requirements:

1. `@kind path-problem` in the metadata.
2. An import of a path-graph module (commonly `import DataFlow::PathGraph` or `import Flow::PathGraph`).
3. A four-column `select` clause of the form
   `select element, source, sink, message`
   where `source` and `sink` are path nodes.

Skeleton (full data-flow configuration is covered in Chapters 8 and 9):

```
/**
 * @name Path query skeleton
 * @description Minimal skeleton showing the structure of a path-problem query.
 * @kind path-problem
 * @id mustlearn/js-path-preview
 * @tags security
 * @problem.severity warning
 * @precision medium
 */
import javascript
import DataFlow::PathGraph

from DataFlow::PathNode source, DataFlow::PathNode sink
where
  // Placeholder: replace with a real flow condition
  source.getNode() = sink.getNode() and
```

```
  false
select sink.getNode(), source, sink, "Placeholder path result"
```

(The complete file is at `examples/chapter06/path-preview.ql`.)

Path results are most useful when viewed in the CodeQL extension for VS Code or in a SARIF viewer; the raw CLI output is harder to interpret.

Choosing a Query Kind

Goal	@kind	Typical select shape
Flag a single location	problem	select element, message
Show a path that explains the issue	path-problem	select element, source, sink, message
Measure something	metric	select number or select entity, number
Diagnostic / tooling feedback	diagnostic	varies

Start with `@kind problem` for most custom checks. Move to `path-problem` when the value of the result depends on seeing the path.

Next Steps

You can now write complete, metadata-rich queries that produce alerts or metrics. In the next chapter we explore the standard language libraries that give you the AST, control flow, and other building blocks you need to write useful queries against real codebases.

All examples referenced above live in the companion repository:
github.com/rod-trent/MustLearnCodeQL
under `examples/chapter06/`.

Must Learn CodeQL — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL